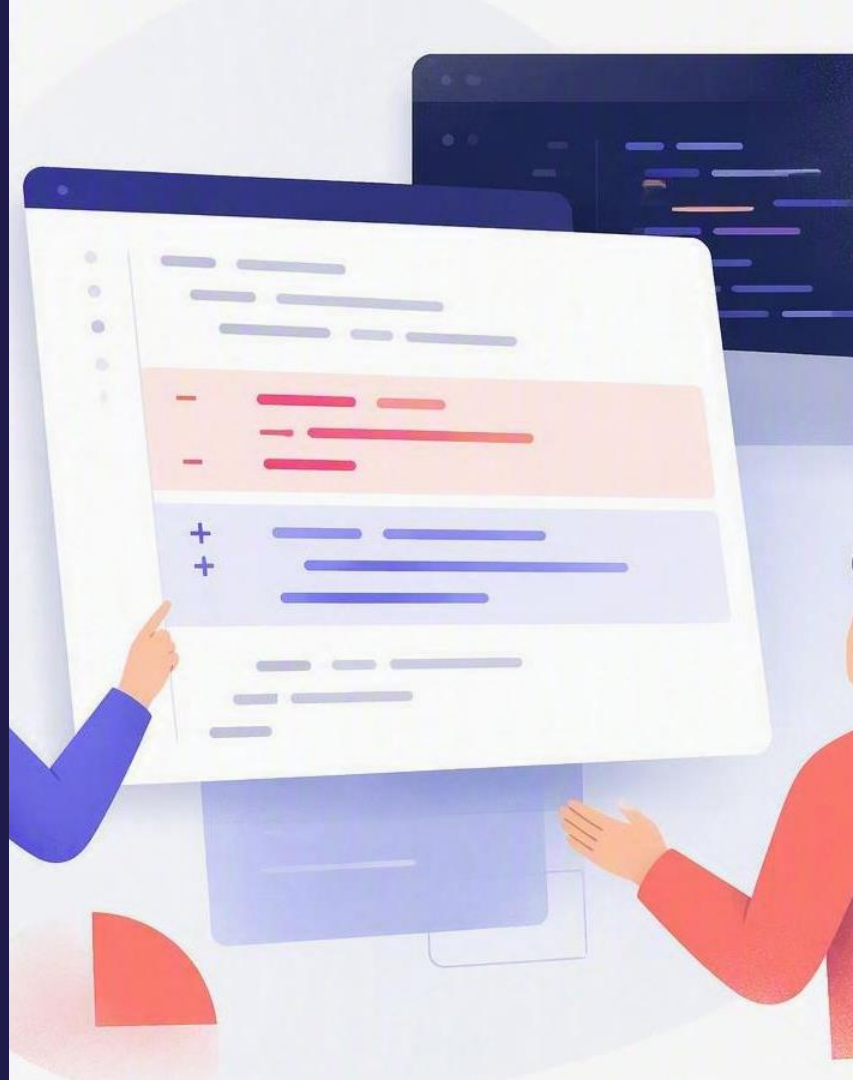

Revisiones de código con GitHub Copilot

Acelerar las revisiones, elevar la calidad y reducir la carga cognitiva del equipo

Capacitación técnica para equipos de desarrollo



INTRODUCCIÓN

Por qué las revisiones se vuelven un cuello de botella

Ciclos largos

Las revisiones se acumulan y retrasan la combinación de cambios.

Comentarios incoherentes

Cada revisor aplica criterios distintos entre equipos y repositorios.

Sugerencias poco accionables

Cuesta convertir una observación en un cambio concreto de código.

Errores desapercibidos

Problemas de estilo y seguridad que llegan a la rama principal.

Un revisor colaborativo, no un reemplazo

Copilot actúa como asistente de revisión: el juicio final y la aprobación siguen siendo humanos.

Resultado buscado: revisiones más rápidas, mayor calidad y menos carga cognitiva para el equipo.

- Detectar problemas y sugerir mejoras concretas.
- Redactar resúmenes y comentarios de revisión.
- Detectar y corregir vulnerabilidades.
- Aplicar instrucciones personalizadas de revisión.
- Mantener patrones y estándares coherentes entre equipos y repositorios.
- Revisar código en GitHub.com y localmente en el IDE.

Qué vas a poder hacer al terminar

- 1 Explicar cómo Copilot simplifica las revisiones y las solicitudes de cambios.
 - 2 Identificar sus principales características de revisión.
 - 3 Solicitar e interpretar revisiones en GitHub.com y comprender sus límites.
 - 4 Ejecutar revisiones locales en el IDE con instrucciones personalizadas.
 - 5 Utilizar PRU para análisis más profundos y con mayor contexto.
 - 6 Automatizar revisiones entre repositorios con conjuntos de reglas.
 - 7 Aplicar las sugerencias con juicio humano y pruebas.
-

PRERREQUISITOS

Lo que necesitás antes de empezar

01

Cuenta de GitHub

Acceso a los repositorios donde vas a revisar cambios.

02

Copilot habilitado

Pro, Pro+, Business o Enterprise para las funcionalidades completas.

03

Bases de revisión

Conocimientos de solicitudes de cambios y revisión de código.

04

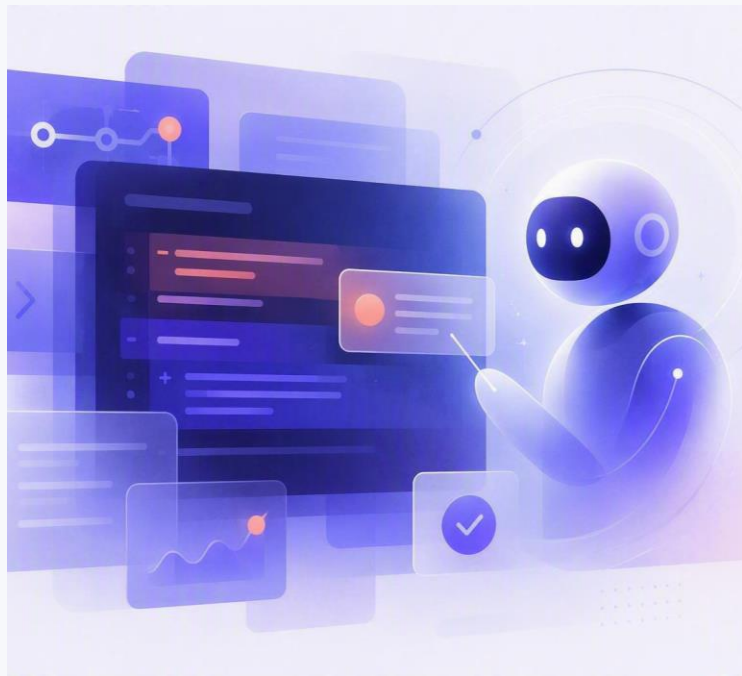
IDE (recomendado)

Visual Studio Code o IDE de JetBrains para revisiones locales.

Las funcionalidades completas de revisión requieren un plan de Copilot compatible.

Lo que Copilot agrega a la revisión

- Detecta problemas comunes antes que un humano.
- Redacta comentarios de revisión claros.
- Resume solicitudes de incorporación de cambios.
- Detecta riesgos de seguridad.
- Aplica directrices personalizadas del equipo.
- Revisa código dentro del IDE.
- Trabaja con distintos lenguajes y convenciones.



CARACTERÍSTICAS CLAVE

Cinco capacidades que usarás a diario

Resúmenes de cambios

Descripción automática con el resumen y los archivos afectados.

Correcciones de seguridad

Integración con el análisis de código para detectar vulnerabilidades.

Explicaciones línea a línea

Seleccionar código y pedir que explique su funcionamiento.

Redacción de comentarios

Comentarios accionables según procedimientos recomendados.

Revisiones en el IDE

Detectar y solucionar problemas antes de abrir la solicitud.

EJEMPLO

De una entrada sin sanitizar a un parche seguro

Detecta

Una entrada sin sanitizar utilizada con `eval()` en el código de la solicitud de cambios.

Sugiere

Reemplazar por un analizador seguro como `JSON.parse()`, con un parche alineado a las directrices del repositorio.

El mismo patrón aplica a otros lenguajes: por ejemplo, reemplazar `exec()` por `subprocess.run()` en Python.

Cinco formas en que Copilot ayuda en las revisiones

01

Sugerencias de revisión de código

02

Revisiones en varios lenguajes

03

Formateo de datos en solicitudes de cambios

04

Generación de resúmenes efectivos

05

Explicación y revisión del código

Sugerencias desde la vista Archivos modificados

- Seleccionar una línea o bloque de código en la solicitud de cambios.
- Pedir a Copilot que sugiera mejoras, detecte problemas o proponga refactorizaciones.
- Incorporar la propuesta al comentario junto con la explicación del revisor.

Ejemplo

Ante lógica repetida en Ruby, Copilot propone una refactorización más limpia que el revisor puede adjuntar al comentario.

La responsabilidad de la revisión sigue siendo humana; Copilot genera comentarios accionables.

Varios lenguajes y formato coherente

Revisión en varios lenguajes

- Detecta desvíos de los procedimientos recomendados o de las directrices del equipo.
- Propone mejoras alineadas con las convenciones de cada lenguaje.
- Permite comentar con precisión aunque el revisor no domine el lenguaje.

Formateo de datos

- Detecta tablas mal formateadas en las descripciones de las solicitudes.
- Propone una versión corregida según las directrices de Markdown de la empresa.
- Aplica las instrucciones de `.github/copilot-instructions.md` en línea.

Copilot como revisor asignado

- Se agrega desde el menú Revisores de la solicitud de cambios.
- Sus comentarios no aprueban ni rechazan: no bloquean las combinaciones.
- Se comportan como los de cualquier revisor: reacciones, resolución y respuestas.
- Señala tipos faltantes, problemas de formato y posibles errores.

< 30 s

duración habitual de una revisión de Copilot

Su función es dar contexto y sugerencias a los revisores humanos, no emitir un veredicto.

PROCESO

Seis pasos para revisar con Copilot en GitHub.com

1 Abrir la solicitud

Crear o abrir la solicitud de incorporación de cambios.

2 Agregar Copilot

Asignarlo desde el menú Revisores.

3 Esperar la revisión

Suele completarse en menos de 30 segundos.

4 Revisar comentarios

Leer las observaciones sobre cada línea de código.

5 Aplicar sugerencias

Pedir soluciones cuando haga falta.

6 Validar los cambios

Ejecutar las pruebas correspondientes.

Ejemplo de solicitud: "Sugerir una corrección para este comentario de revisión: reemplace exec() por una función más segura."

Copilot tiene un papel de asesor

Lo que no hace

- No aprueba ni rechaza solicitudes de cambios.
- Sus comentarios no cuentan para las aprobaciones requeridas.
- No reemplaza a los revisores humanos.

Para qué usarlo

- Detectar problemas y generar sugerencias accionables.
- Acelerar comprobaciones rutinarias y repetitivas.
- Liberar tiempo para las decisiones complejas.

Las decisiones arquitectónicas, los matices y la aprobación final son responsabilidad humana.

Revisiones locales en el IDE

- Revisar en Visual Studio Code o IDE de JetBrains antes de abrir la solicitud.
- Detectar tempranamente infracciones de estilo, problemas de seguridad e incumplimientos de buenas prácticas.
- Definir reglas en `.github/copilot-instructions.md`: foco en seguridad, sin interpolaciones inseguras, docstrings con parámetros y valores devueltos.
- Analizar diferencias más grandes con contexto alineado al estilo del repositorio.

Caso de uso

Un desarrollador agrega código repetitivo en un servicio TypeScript. Copilot sugiere extraer una función auxiliar y el problema se corrige antes de enviar el código, reduciendo el ruido de la revisión posterior.

Reglas específicas por rutas del repositorio

- 1 Crear la carpeta `.github/instructions` en la raíz del repositorio.
- 2 Agregar uno o varios archivos terminados en `.instructions.md`.
- 3 Definir las rutas con `applyTo` usando sintaxis glob.
- 4 Escribir las instrucciones en Markdown debajo del frontmatter.

Copilot aplica estas instrucciones en las revisiones y en la generación de código de las rutas correspondientes.

Ejemplos de rutas

`app/models/**/*.rb`

modelos Ruby

`**/*.ts, **/*.tsx`

archivos TypeScript

`**`

todo el repositorio

Automatizar revisiones con conjuntos de reglas

- Asignar Copilot automáticamente a las solicitudes dirigidas a ramas protegidas.
- Combinar la revisión con comprobaciones de estado complementarias.
- Revisar de forma coherente incluso cambios pequeños y actualizaciones de dependencias.

Copilot

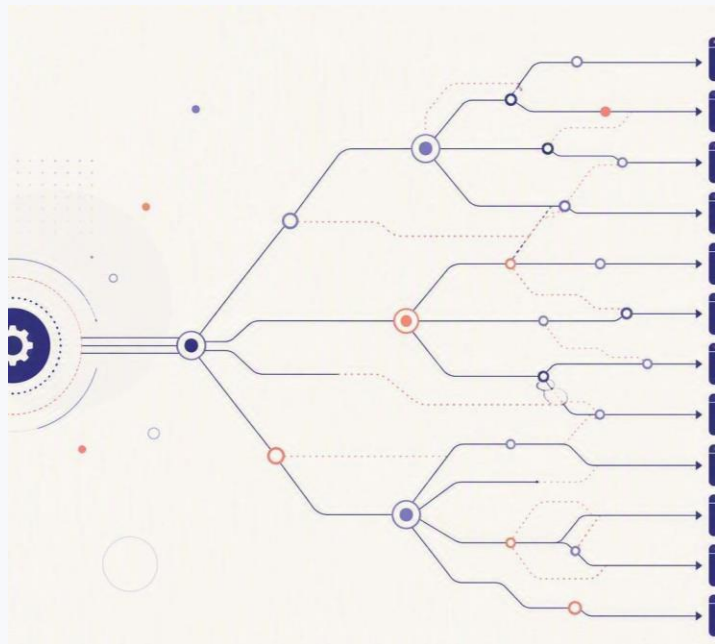
estilo y legibilidad

Code scanning

vulnerabilidades

Pruebas

funcionalidad



Revisiones automáticas: cuenta, repositorio y organización

Cuenta

Copilot Pro o Pro+

- Perfil → Su Copilot.
- Buscar la revisión de código automática.
- Seleccionar Habilitado.

Repositorio

Administradores

- Configuración → Código y automatización → Reglas.
- Nuevo conjunto de reglas de rama, estado Activo.
- Requerir solicitud de cambios y revisión de Copilot.

Organización

Propietarios

- Configuración → Repositorio → Conjuntos de reglas.
- Incluir o excluir repositorios y ramas de destino.
- Exigir solicitud de cambios y revisión de Copilot.

PRU

Unidades de solicitud Premium: qué son y qué habilitan

Capacidad adicional de procesamiento y contexto para las tareas de revisión más exigentes.

- Analizar diferencias completas y de gran tamaño.
- Interpretar directrices personalizadas del repositorio.
- Generar correcciones accionables y explicativas.
- Comprobar seguridad, estilo y buenas prácticas en profundidad.

Las tareas rutinarias y ligeras pueden no consumir PRU.



IMPACTO EN EL EQUIPO

Por qué las PRU importan

Análisis más profundo

Vulnerabilidades sutiles, lógica duplicada e infracciones de estilo en diferencias grandes.

Coherencia

Las mismas comprobaciones de seguridad, legibilidad y estilo en cada revisión.

Control de ráfagas

Mantener la calidad en picos de actividad mientras los humanos deciden lo complejo.

Ejemplo · microservicios

Con Go, Python y TypeScript, Copilot revisa cada servicio según las buenas prácticas de su lenguaje: detecta una llamada peligrosa a `eval()` en JavaScript y señala una falta de comprobación de errores en Go.

Cómo evaluar las revisiones basadas en PRU

Plazo de solicitud

Tiempo desde la apertura hasta la combinación.

Indicadores de calidad

Menos problemas de estilo o seguridad tras la combinación.

Experiencia del equipo

Percepción de rapidez y claridad de las revisiones.

3 días → 1 día

Tiempo de combinación de solicitudes grandes tras habilitar revisiones con PRU, con menos confirmaciones de seguimiento.

Cuatro tácticas para usar bien las PRU

Planificar por adelantado

Alertas al 75 %, 90 % y 100 % del uso mensual.

Usarlas estratégicamente

Reservarlas para cambios grandes o de alto riesgo.

Refinar los mensajes

Solicitudes claras y específicas reducen reintentos.

Escalar el plan

Considerar un plan superior si el consumo crece de forma sostenida.

–30 % de consumo mensual en un equipo que reservó las revisiones premium para código de producción, sin perder calidad.

Idea fundamental

Las PRU hacen posibles las funcionalidades avanzadas de revisión. Comprender su funcionamiento, medir su impacto y optimizar su uso permite realizar revisiones más profundas y contextuales, y escalar la calidad en toda la organización.

Más profundidad

Análisis contextual de diferencias grandes.

Más escala

Estándares coherentes entre repositorios.

Juicio humano

La aprobación final sigue siendo de las personas.